Operating Systems

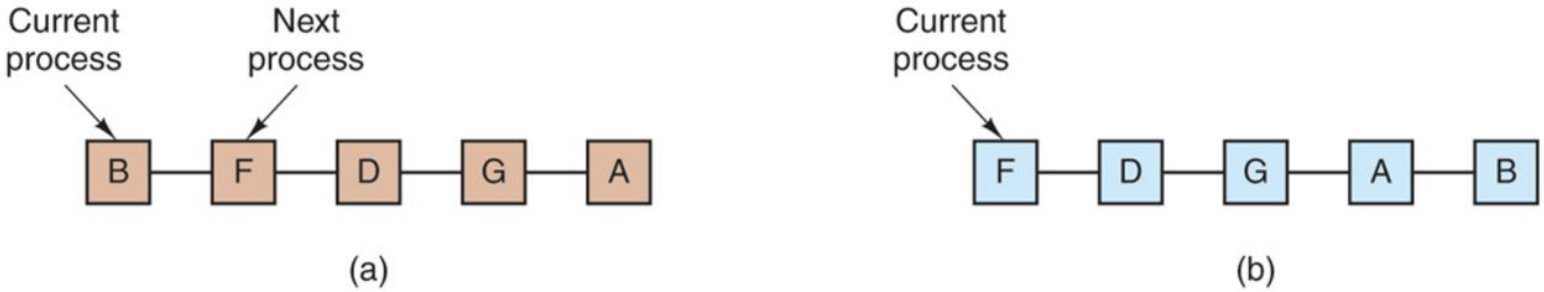
Rowan University

Overview

1. Interprocess Communication
2. Scheduler Review
3. Domain Separation, Encapsulation

Round Robin Scheduler

Figure 2-43

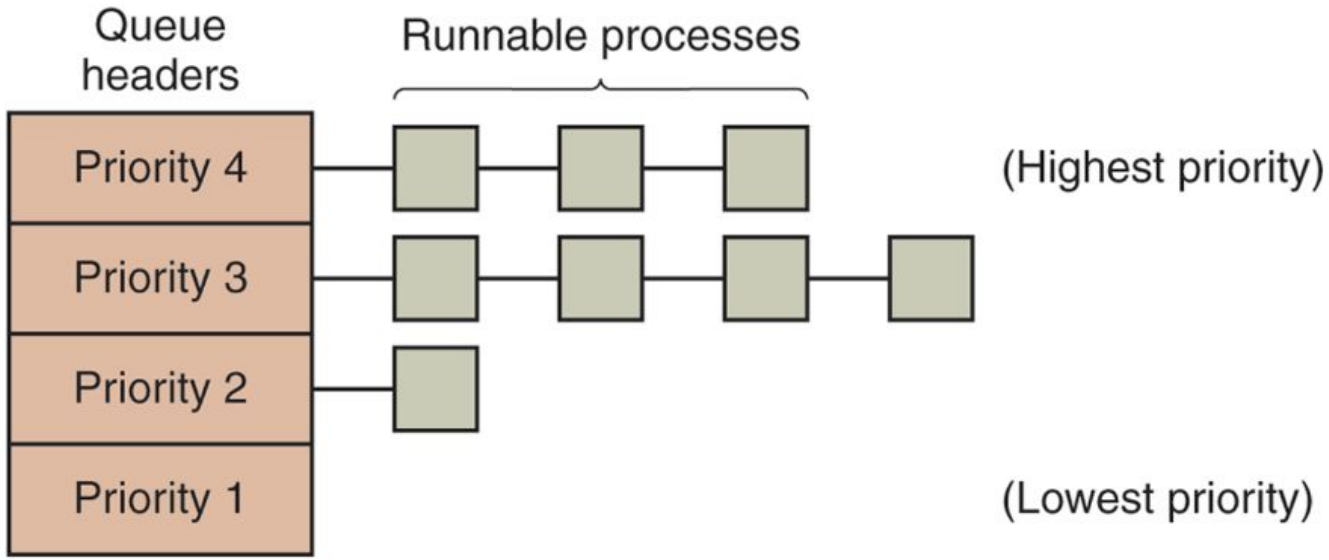


Round-robin scheduling. (a) The list of runnable processes. (b)The list of runnable processes after *B* uses up its quantum.

Priority Scheduling

- Priority Queue of Queues
- Highest Priority
- Then FIFO for all processes at that priority

Figure 2-44



A scheduling algorithm with four priority classes.

Process Table

```
43    struct procent {        /* Entry in the process table       */
44        uint16  prstate;    /* Process state: PR_CURR, etc.     */
45        pri16   prprio;     /* Process priority                 */
46        char    *prstkptr;  /* Saved stack pointer              */
47        char    *prstkbase; /* Base of run time stack           */
48        uint32  prstklen;   /* Stack length in bytes            */
49        char    prname[PNMLEN]; /* Process name                 */
50        sid32   prsem;      /* Semaphore on which process waits */
51        pid32   prparent;   /* ID of the creating process       */
52        umsg32  prmsg;      /* Message sent to this process     */
53        bool8   prhasmsg;   /* Nonzero iff msg is valid         */
54        int16   prdesc[NDESC];  /* Device descriptors for process   */
55    };
```

Queue

initialize.c

```
/* Create a ready list for processes */

readylist = newqueue();
```

```
/*------------------------------------------------------------------------
 *  newqueue  -  Allocate and initialize a queue in the global queue table
 *------------------------------------------------------------------------
 */
qid16   newqueue(void)
{
    static qid16    nextqid=NPROC;  /* Next list in queuetab to use */
    qid16       q;      /* ID of allocated queue    */

    q = nextqid;
    if (q >= NQENT) {       /* Check for table overflow */
        return SYSERR;
    }

    nextqid += 2;           /* Increment index for next call*/

    /* Initialize head and tail nodes to form an empty queue */

    queuetab[queuehead(q)].qnext = queuetail(q);
    queuetab[queuehead(q)].qprev = EMPTY;
    queuetab[queuehead(q)].qkey  = MAXKEY;
    queuetab[queuetail(q)].qnext = EMPTY;
    queuetab[queuetail(q)].qprev = queuehead(q);
    queuetab[queuetail(q)].qkey  = MINKEY;
    return q;
}
```

Set LL Head

Set LL Tail

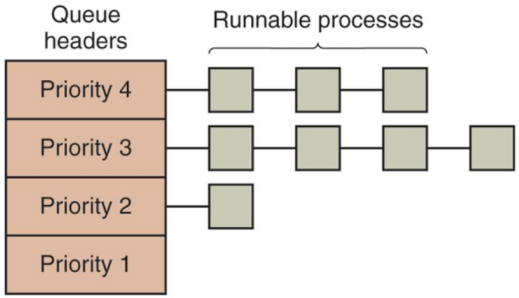
Ready

```
7   /*------------------------------------------------------------------------
8    *  ready  -  Make a process eligible for CPU service
9    *------------------------------------------------------------------------
10   */
11  status  ready(
12      pid32     pid     /* ID of process to make ready  */
13      )
14  {
15      register struct procent *prptr;
16
17      if (isbadpid(pid)) {
18          return SYSERR;
19      }
20
21      /* Set process state to indicate ready and add to ready list */
22
23      prptr = &proctab[pid];
24      prptr->prstate = PR_READY;
25      insert(pid, readylist, prptr->prprio);
26      resched();
27
28      return OK;
29  }
```

- Ready is not running
- Marking a process ready inserts it into the readylist at position = priority

Insert

qkey = Priority




```
22      curr = firstid(q);
23      while (queuetab[curr].qkey >= key) {
24          curr = queuetab[curr].qnext;
25      }
```

```
3   #include <xinu.h>
4
5   /*------------------------------------------------------------------------
6    *  insert  -  Insert a process into a queue in descending key order
7    *------------------------------------------------------------------------
8    */
9   status  insert(
10          pid32     pid,        /* ID of process to insert  */
11          qid16     q,      /* ID of queue to use       */
12          int32     key     /* Key for the inserted process */
13      )
14  {
15      qid16   curr;           /* Runs through items in a queue*/
16      qid16   prev;           /* Holds previous node index    */
17
18      if (isbadqid(q) || isbadpid(pid)) {
19          return SYSERR;
20      }
21
22      curr = firstid(q);
23      while (queuetab[curr].qkey >= key) {
24          curr = queuetab[curr].qnext;
25      }
26
27      /* Insert process between curr node and previous node */
28
29      prev = queuetab[curr].qprev;    /* Get index of previous node   */
30      queuetab[pid].qnext = curr;
        queuetab[pid].qprev = prev;
32      queuetab[pid].qkey = key;
        queuetab[prev].qnext = pid;
        queuetab[curr].qprev = pid;
35      return OK;
36  }
```

Set LL Head

Set LL Tail

Reschedule

- Get the current process (ptold)
- If it's the current process
- Check if it's priority is higher than the highest priority process from readylist
  - firstkey(readylist)
- Otherwise, current process moves to READY
- Pull the next process from the queue and run it

```
/* Inline queue manipulation functions */

#define queuehead(q)    (q)
#define queuetail(q)    ((q) + 1)
#define firstid(q)  (queuetab[queuehead(q)].qnext)
#define lastid(q)   (queuetab[queuetail(q)].qprev)
#define isempty(q)  (firstid(q) >= NPROC)
#define nonempty(q) (firstid(q) <  NPROC)
#define firstkey(q) (queuetab[firstid(q)].qkey)
#define lastkey(q)  (queuetab[ lastid(q)].qkey)
```

Context switch

```
/*------------------------------------------------------------------------
 *  resched  -  Reschedule processor to highest priority eligible process
 *------------------------------------------------------------------------
 */
void    resched(void)       /* Assumes interrupts are disabled  */
{
    struct procent *ptold;  /* Ptr to table entry for old process   */
    struct procent *ptnew;  /* Ptr to table entry for new process   */

    /* If rescheduling is deferred, record attempt and return */

    if (Defer.ndefers > 0) {
        Defer.attempt = TRUE;
        return;
    }

    /* Point to process table entry for the current (old) process */

    ptold = &proctab[currpid];

    if (ptold->prstate == PR_CURR) {  /* Process remains eligible */
        if (ptold->prprio > firstkey(readylist)) {
            return;
        }

        /* Old process will no longer remain current */

        ptold->prstate = PR_READY;
        insert(currpid, readylist, ptold->prprio);
    }

    /* Force context switch to highest priority ready process */

    currpid = dequeue(readylist);
    ptnew = &proctab[currpid];
    ptnew->prstate = PR_CURR;
    preempt = QUANTUM;      /* Reset time slice for process */
    ctxsw(&ptold->prstkptr, &ptnew->prstkptr);

    /* Old process returns here when resumed */

    return;
}
```

Context Switch (ctxsw)

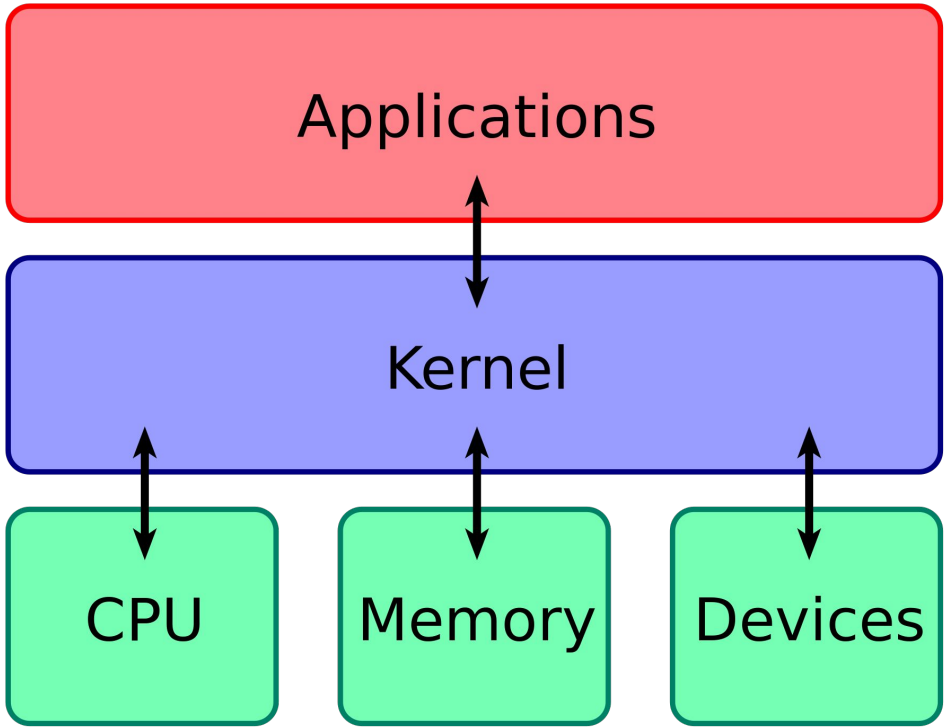
- Written in assembly
- Targets x86 architecture

1. Save old processes stack pointer into memory
2. Load the next instruction as the new processes stack pointer

Stack pointer is pointing to the next instruction to run for a process

```
/* ctxsw.S - ctxsw (for x86) */

        .text
        .globl  ctxsw

/*------------------------------------------------------------------------
 * ctxsw -  X86 context switch; the call is ctxsw(&old_sp, &new_sp)
 *------------------------------------------------------------------------
 */
0 references
ctxsw:
        pushl   %ebp        /* Push ebp onto stack      */
        movl    %esp,%ebp   /* Record current SP in ebp */
        pushfl          /* Push flags onto the stack    */
        pushal          /* Push general regs. on stack  */

        /* Save old segment registers here, if multiple allowed */

        movl    8(%ebp),%eax    /* Get mem location in which to */
                        /*   save the old process's SP  */
        movl    %esp,(%eax) /* Save old process's SP    */
        movl    12(%ebp),%eax   /* Get location from which to   */
                        /*   restore new process's SP   */

        /* The next instruction switches from the old process's */
        /*   stack to the new process's stack.          */

        movl    (%eax),%esp /* Pick up new process's SP */

        /* Restore new seg. registers here, if multiple allowed */

        popal           /* Restore general registers    */
        movl    4(%esp),%ebp    /* Pick up ebp before restoring */
                        /*   interrupts         */
        popfl           /* Restore interrupt mask   */
        add $4,%esp     /* Skip saved value of ebp  */
        ret         /* Return to new process    */

```

User Space and Kernel Space



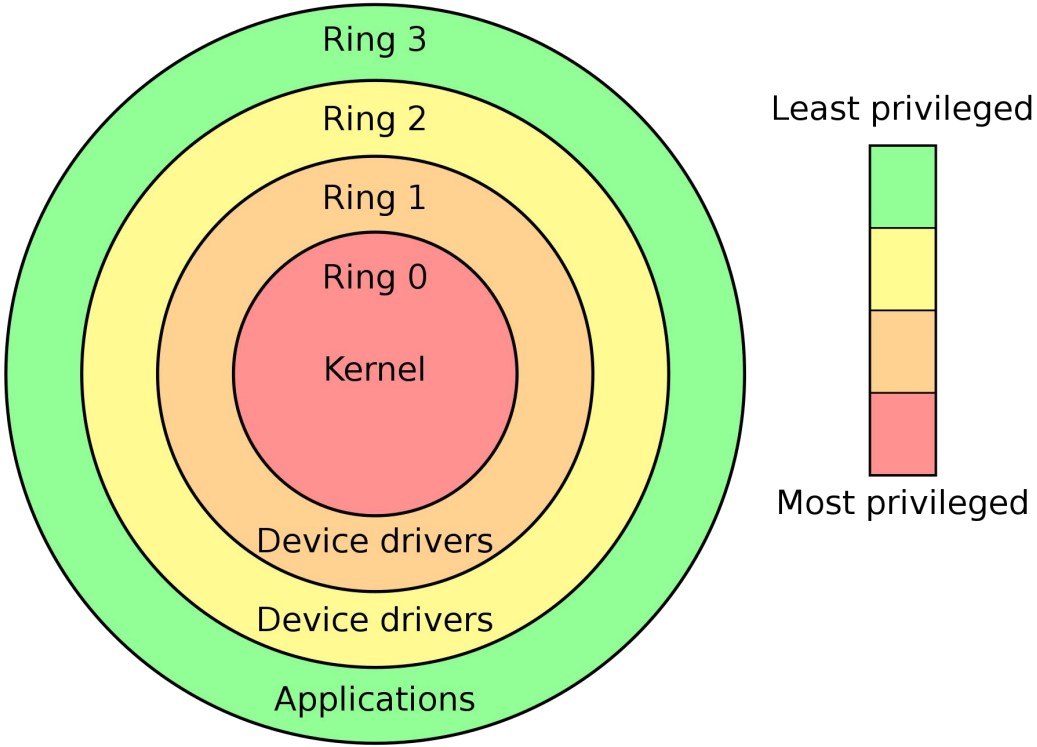
By Bobbo - Own work, CC BY-SA 3.0,
https://commons.wikimedia.org/w/index.php?curid=4392180

User Space and Kernel Space

Various layers within Linux, also showing separation between the userland and kernel space

User mode	**User applications**	*bash*, *LibreOffice*, *GIMP*, *Blender*, *0 A.D.*, *Mozilla Firefox*, ...				
	System components	**init daemon**: *OpenRC*, *runit*, *systemd*...	**System daemons**: *polkitd*, *smbd*, *sshd*, *udevd*...	**Window manager**: *X11*, *Wayland*, *SurfaceFlinger* (*Android*)	**Graphics**: *Mesa*, *AMD Catalyst*, ...	**Other libraries:** *GTK*, *Qt*, *EFL*, *SDL*, *SFML*, *FLTK*, *GNUstep*, ...
	C standard library	`fopen`, `execv`, `malloc`, `memcpy`, `localtime`, `pthread_create` ... (up to 2000 subroutines) *glibc* aims to be fast, *musl* aims to be lightweight, *uClibc* targets embedded systems, *bionic* was written for Android, etc. All aim to be POSIX/SUS-compatible.				
Kernel mode	**Linux kernel**	`stat`, `splice`, `dup`, `read`, `open`, `ioctl`, `write`, `mmap`, `close`, `exit`, etc. (about 380 system calls) The Linux kernel System Call Interface (SCI), aims to be POSIX/SUS-compatible[2]				
		Process scheduling subsystem	IPC subsystem	Memory management subsystem	Virtual files subsystem	Networking subsystem
		Other components: ALSA, DRI, evdev, klibc, LVM, device mapper, Linux Network Scheduler, Netfilter Linux Security Modules: *SELinux*, *TOMOYO*, *AppArmor*, *Smack*				
Hardware (CPU, main memory, data storage devices, etc.)						

Protection Rings



By Hertzsprung at English Wikipedia, CC BY-SA 3.0,
https://commons.wikimedia.org/w/index.php?curid=8950144